Panduan Membuat Fitur — Web

Manote · Next.js, TypeScript, Tailwind, TanStack Query · contoh acuan: fitur login

1. Gambaran alur

Setiap fitur web mengikuti urutan yang sama. Halaman hanya merangkai komponen; komponen mengambil data lewat hook; hook memanggil fungsi API.

Halaman (app/) → **Komponen** → **Hook** → **Fungsi API** → **Backend**

Lapisan	Tugasnya	Contoh di fitur login
Halaman	Menentukan URL. Isinya sesedikit mungkin.	`app/login/page.tsx`
Komponen	Tampilan dan interaksi. Diberi `"use client"` jika memakai state atau hook.	`login-form.tsx`
Hook	Membungkus TanStack Query: `useQuery` untuk membaca, `useMutation` untuk mengubah.	`use-login.ts`, `use-current-user.ts`
Fungsi API	Tipe data dan pemanggilan `apiFetch`.	`auth-api.ts`
API client	Menambah alamat backend, header token, dan membaca pesan error.	`lib/api-client.ts`

2. Peta file fitur login

```
web/src/
├── app/
│   ├── page.tsx                    mengalihkan ke /dashboard
│   ├── login/page.tsx              halaman /login
│   └── dashboard/page.tsx          halaman /dashboard
├── features/
│   ├── auth/
│   │   ├── api/auth-api.ts         tipe User, login(), getCurrentUser()
│   │   ├── hooks/                  use-login, use-current-user, use-logout
│   │   ├── components/login-form.tsx
│   │   └── lib/token-storage.ts    simpan token di localStorage
│   └── dashboard/components/dashboard.tsx
├── lib/api-client.ts
└── providers/query-provider.tsx
```

3. Langkah membuat fitur baru

Contoh yang dipakai: fitur **edit profil**, dengan endpoint backend `PUT /api/profile`.

Langkah 1 — Pastikan kontrak API

Lihat panduan backend. Yang dibutuhkan web: method, URL, bentuk request, bentuk response.

Langkah 2 — Fungsi API dan tipe

```
// src/features/profile/api/profile-api.ts
import { apiFetch } from "@/lib/api-client";
import type { User } from "@/features/auth/api/auth-api";

export type UpdateProfileRequest = { fullName: string };

export function updateProfile(request: UpdateProfileRequest) {
  return apiFetch<User>("/api/profile", {
    method: "PUT",
    body: JSON.stringify(request),
  });
}
```

`apiFetch` sudah otomatis mengirim token dan membuka bungkus response: yang dikembalikan adalah isi `data`, jadi tipe yang ditulis cukup tipe datanya (di sini `User`).

Langkah 3 — Hook

```
// src/features/profile/hooks/use-update-profile.ts
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { currentUserQueryKey } from "@/features/auth/hooks/use-current-user";
import { updateProfile } from "../api/profile-api";

export function useUpdateProfile() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: updateProfile,
    onSuccess: (user) => {
      queryClient.setQueryData(currentUserQueryKey, user);
    },
  });
}
```

> **Aturan cache:** setelah data berubah, perbarui cache-nya di `onSuccess`, dengan `setQueryData` (jika response berisi data baru) atau `invalidateQueries` (agar diambil ulang). Tanpa ini, layar lain masih menampilkan data lama.

Langkah 4 — Komponen

```
// src/features/profile/components/profile-form.tsx
"use client";

import { useState } from "react";
import { useUpdateProfile } from "../hooks/use-update-profile";

export function ProfileForm({ initialFullName }: { initialFullName: string }) {
  const updateProfile = useUpdateProfile();
  const [fullName, setFullName] = useState(initialFullName);

  function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    updateProfile.mutate({ fullName });
  }

  return (
    <form onSubmit={handleSubmit} className="flex flex-col gap-4">
      <input value={fullName} onChange={(e) => setFullName(e.target.value)} required />
      {updateProfile.isError && <p role="alert">{updateProfile.error.message}</p>}
      {updateProfile.isSuccess && <p>Profil tersimpan.</p>}
      <button type="submit" disabled={updateProfile.isPending}>
        {updateProfile.isPending ? "Menyimpan..." : "Simpan"}
      </button>
    </form>
  );
}
```

Tiga keadaan yang selalu ditangani: sedang memproses (`isPending`), gagal (`isError`), dan berhasil.

Langkah 5 — Halaman

Nama folder di dalam `app/` menjadi URL. Folder `app/profile/` menghasilkan halaman `/profile`.

```
// src/app/profile/page.tsx
import { ProfilePage } from "@/features/profile/components/profile-page";

export default function Page() {
  return <ProfilePage />;
}
```

Langkah 6 — Proteksi halaman dan navigasi

Halaman yang butuh login memakai pola yang sama dengan `dashboard.tsx`: panggil `useCurrentUser()`, dan alihkan ke `/login` jika gagal.

```
const { data: user, isPending, isError } = useCurrentUser();

useEffect(() => {
  if (isError) router.replace("/login");
}, [isError, router]);

if (isPending || isError) return <p>Memuat...</p>;
return <ProfileForm initialFullName={user.fullName} />;
```

Terakhir, tambahkan tautan ke halaman baru dari dashboard dengan komponen `Link` dari `next/link`.

Langkah 7 — Jalankan dan uji

```
cd web
npm run dev        # buka http://localhost:3000 (backend harus menyala di port 8080)
npm run lint
npm run build      # memastikan tidak ada error TypeScript
```

---

4. Penanganan error

Backend selalu menjawab dengan bentuk `{ status, title, code, data }`. Jika `status` bernilai `false`, `apiFetch` melempar `ApiError` yang membawa `code` dan pesan dari `title`.

Aturannya: setiap error memunculkan **toast** secara otomatis. Fitur baru tidak perlu menulis apa pun untuk itu. Pengecualiannya hanya jika layar menampilkan error-nya sendiri.

Kasus	Yang terjadi otomatis	Yang Anda tulis di fitur
Error dari aksi pengguna (hapus, simpan)	Toast berisi pesan dari `title`	Tidak ada
Layar menampilkan error sendiri (pesan di form, daftar dengan Coba lagi, indikator simpan)	Tidak ada toast	Beri `{ handlesError: true }` pada `apiFetch`, lalu tampilkan `error.message`
Lebih dari 30 detik tanpa jawaban	Permintaan dihentikan dan toast muncul, walaupun `handlesError` diisi	Tidak ada
Sesi berakhir (4012)	Token dibuang dan pengguna dialihkan ke `/login` dengan pesan Sesi berakhir	Tidak ada
Butuh tindakan khusus per kode	—	Bandingkan `error.code` dengan `ApiCode`

```
// Bawaan: gagal -> toast otomatis
return apiFetch<void>(`/api/notes/${id}`, { method: "DELETE" });

// Layar menampilkan error sendiri -> tanpa toast
return apiFetch<Note[]>("/api/notes", undefined, { handlesError: true });
```

```
import { ApiError } from "@/lib/api-client";
import { ApiCode } from "@/lib/api-codes";

if (error instanceof ApiError && error.code === ApiCode.NOT_FOUND) {
  router.replace("/notes");
}
```

Hal yang perlu diingat

- Browser memanggil `/api` di alamat web itu sendiri, lalu server web meneruskannya ke backend (`rewrites` di `next.config.ts`). Alamat backend diatur lewat `MANOTE_BACKEND_URL`.
- Daftar kode ada di `src/lib/api-codes.ts` dan harus sama dengan `ErrorCode` di backend.
- Next.js versi ini punya perubahan dari versi lama. Jika ragu soal sebuah API, baca dokumentasi di `node_modules/next/dist/docs/`.

---

5. Checklist fitur web

- ☐ Tipe dan fungsi API dibuat di `features/<fitur>/api/`.
- ☐ Hook dibuat di `features/<fitur>/hooks/`; cache diperbarui setelah mutasi.
- ☐ Komponen menangani keadaan memproses, gagal (menampilkan `error.message`), dan berhasil.
- ☐ Halaman dibuat di `app/<url>/page.tsx` dan diproteksi jika butuh login.
- ☐ Ada tautan menuju halaman baru.
- ☐ `npm run lint` dan `npm run build` lolos; fitur dicoba di browser.